# Compilation, energy, and exam preparation

Compilation, energy, exam preparation, project defense

Dinu-Ștefan Rusu MEC

# What you should leave with

### Compilation strategies

Interpretation, just-in-time and ahead-of-time compilation, and what each one trades away.

### Build modes

Why Dart ships two compilers, what Android's runtime does, and why iOS allows no choice.

### Energy use

Race to idle, thermal throttling, and the duty cycle that decides a device's battery life.

### Exam preparation

The exam format, what to revise per lecture, the project defense, and the concept presentation.

1

# Compilation strategies

Interpretation, JIT, AOT, mobile runtimes

Contents Page

1 **Compilation strategies** 3
2 Energy at the platform level 13
3 Exam preparation 19
4 Defense and presentation 31

# Interpretation, JIT and AOT

| | Interpretation | JIT | AOT |
|---|---|---|---|
| Mechanism | run bytecode, one instruction at a time | compile hot paths while the program runs | compile everything before the program runs |
| Start-up | instant | slow: a compiler runs | instant: already native |
| Peak speed | lowest | highest: real types | high: no run-time data |
| Memory | small | largest at run time | smallest at run time |

**Start-up, peak speed and memory trade against each other.** A phone needs all three at once, on a battery, so no shipping runtime is pure.

# From source text to machine code

**Dart**
The front end parses your code into kernel, an intermediate representation with no syntax left in it.

In debug, the VM compiles kernel to machine code as functions are first called. In release, the ahead-of-time compiler turns kernel into ARM machine code and packs it into a snapshot inside the app.

**TinyGo**
The ordinary Go front end, then LLVM intermediate representation, then LLVM emits code for the target.

Cortex-M, RISC-V, AVR and WebAssembly are all just back ends. TinyGo is a different compiler for the same language, not a different language.

**Every toolchain has the same three stages.** Front end, intermediate representation, back end. What differs between them is only when the back end runs.

# Dart build modes

```
flutter run                # debug
#  JIT in the Dart VM, hot reload on
flutter run --profile      # profile
#  AOT plus tracing hooks: measure here
flutter build appbundle --release
#  AOT to ARM, nothing left to reload
```

Every performance claim names the build mode it came from.

**JIT or AOT**

”Dart is a JIT language” and ”Dart is an AOT language” are both half true. It is one language with two compilers, and the build mode picks one.

Hot reload exists because the debug VM can compile new functions into a running isolate.

**JIT for the developer loop, AOT for the user.** Profile mode is the compromise: release code plus the hooks DevTools needs.

# JIT advantages over AOT

- It sees the types that actually arrive. A call site that could take ten classes but only ever takes one is compiled as if it took one.
- It inlines across that assumption, then guards it. If the assumption breaks, the code is thrown away and recompiled.
- It can recompile a hot loop with better information after the program has been running for a while.

**⚠ What it costs**

The compiler runs inside your process: its CPU time, its memory and its pauses are charged to the user. Every cold start pays the warm-up again.

**Phones are launched cold, often, on a battery.** That is why the platforms kept moving compilation earlier instead of making the run-time compiler better.

# Android runtime evolution

| Runtime | When | What it did |
|---|---|---|
| Dalvik | early Android | a trace JIT: it compiled while you scrolled, and janked |
| ART, first release | Android 5 and 6 | ahead of time at install: fast, but large binaries |
| ART today | Android 7 and later | an interpreter, a JIT, and background compilation when idle |
| Baseline profiles | Android 7 and later | hot paths compiled from a profile, at install time since Android 9 |

**Every step moved work earlier.** Run time became install time, and install time became build time.

# iOS and write XOR execute

**The rule**
A memory page may be writable or executable, never both at the same time.

It is the defense that stops an attacker who can write bytes into a process from also running them. A JIT needs exactly that forbidden combination: write machine code into a page, then jump into it.

**The consequences**
Third-party apps do not get the entitlement, so everything you ship to iOS is native code compiled ahead of time.

WebKit's JavaScript engine is the privileged exception. Bitcode is not a workaround: Apple deprecated it in Xcode 14, and a guide that tells you to enable it is out of date.

**Write XOR execute rules out a JIT in third-party iOS apps.** No Dalvik-style runtime on iOS, no hot reload in a release build, and JavaScript that only runs fast inside Safari.

# Cross-platform runtimes in 2026

**Flutter**
Dart compiles ahead of time to native ARM for release, so there is no interpreter in production.

Impeller precompiles its shaders when the engine is built, so no shader is compiled on the user's device. Debug keeps the JIT, which hot reload needs.

**React Native**
Hermes precompiles JavaScript to bytecode at build time, which removes the parse cost at launch.

The bridgeless architecture, with JSI and Fabric, became the default in React Native 0.76 and the legacy one has since been removed. The serialized-JSON bridge is gone: JavaScript calls C++ directly.

**Both ecosystems moved the same work from run time to build time.** Kotlin Multiplatform starts from the other end: shared Kotlin compiled natively, native UI on top.

# Moving work from run time to build time

### Shaders

Impeller precompiles its shaders in the engine, so the first frame with an effect costs what the thousandth costs.

### Bytecode

Hermes precompiles JavaScript, so launch does not include a parser reading the bundle.

### Profiles

A baseline profile compiles the hot paths before the first launch, not during it.

### Tree shaking

Unused code and icons are removed by the release build, not skipped at run time.

### Code generation

Generated protobuf and JSON code replaces reflection, which a small runtime may lack.

### Constants

A const constructor is canonicalized at compile time, so one instance is reused.

# Cold start sequence

1. The OS creates the process and maps the binary.
2. The runtime loads the compiled snapshot.
3. `main()` runs and the first widget tree is built.
4. The first frame is laid out and rasterized.

**What makes it slow**

Work placed before `runApp`, plugins initialized synchronously, a large image decoded on the first frame, a database opened on the UI isolate.

**With an ahead-of-time build, cold start is mostly your code.** The compiler already did its part. Move everything that is not needed for the first screen off the startup path.

2

# Energy at the platform level

Race to idle, thermal throttling, duty cycles

Contents

| | | Page |
|---|---|---|
| 1 | Compilation strategies | 3 |
| **2** | **Energy at the platform level** | 13 |
| 3 | Exam preparation | 19 |
| 4 | Defense and presentation | 31 |

# Race to idle

### Idle is a real state

A modern SoC has deep low-power states, but only when nothing keeps a core awake.

### Frequency scales

The governor raises the clock under load and drops it again. A short burst at a high clock usually beats a long dribble at a low one.

### Wake-ups are expensive

A timer that fires every second, a poll loop, a busy animation: each costs more than the work it performs.

**The practical rule**

Do the work, then let the hardware sleep.

---

**Anything that prevents idle costs more than the work it does.** That includes a run-time compiler warming up.

# Thermal throttling

- Inefficient code raises load, load raises temperature, and the OS throttles the SoC to cool it down.
- So the same code is measurably slower a minute later, on the same device, with nothing else changed.
- Peak performance is what a benchmark reports. Sustained performance is what your user gets.

**Test on a warm phone**

Run the scenario for several minutes before you record a number, and say in your report how long the device had been working.

**Heat is the feedback loop between energy and speed.**

# Energy cost of compilation

### A run-time compiler

Every cycle it spends is a cycle the user's battery pays for, and it spends them again on every cold start.

### A run-time shader

Compiling an effect the first time it appears costs a frame the user sees and energy nobody planned for.

### A build-time decision

Paid once, on your machine. It costs the user nothing on any launch.

**The same question, twice**

How much of this program's behavior can be decided before the user's device is involved.

**Build time is free to the user. Run time is not.**

# Duty cycling at the device level

On an ESP32-class part the spread between deep sleep and radio transmit is roughly four orders of magnitude. The question is not how fast the code is, but what fraction of the time the radio is on.

**Average current**

Average current: active current times the active fraction, plus sleep current times the rest.

1. Wake, on a timer or a sensor interrupt.
2. Sample and batch several readings.
3. Connect, publish, close. Keep it short.
4. Deep sleep, radio off, RAM retained.

**Sleep between publishes instead of staying connected.**

# Measurement methodology

### What

Average current on the board, energy per sync, frame build time, payload size, or publish-to-display latency.

### How

A USB power meter on the board. Profile mode on a real phone, never debug. Many repetitions, never a single shot.

### Report

The value with its unit, the method, the device, the build mode, and the conditions.

**Project requirement five**

One number your team measured, with the method written down. Energy, latency, payload size and frame time all qualify.

**Report the method with every number.**

3

# Exam preparation

Format, revision plan, example questions

Contents

| | | Page |
|---|---|---|
| 1 | Compilation strategies | 3 |
| 2 | Energy at the platform level | 13 |
| **3** | **Exam preparation** | 19 |
| 4 | Defense and presentation | 31 |

# The exam format

- Fifteen multiple-choice questions, 2 points each, 30 points on the paper. The exam is worth 40 of the 100 course points.
- Every question has zero or more correct answers. You score only when all correct options are marked and no incorrect option is.
- There is no partial credit. Two variants, one per row. A non-programmable calculator is allowed.

**Only the grid is graded**

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| A | X | | X | | X |
| B | | X | X | | |
| C | X | | | X | X |
| D | | X | | X | |
| E | | | X | | X |

**Columns are questions, rows are options.** Marks made next to the questions themselves are ignored. To change an answer, fill the wrong box completely and mark the intended one.

# Revision topics, Lectures 1 to 7

| L | Topic | Revise |
|---|---|---|
| 1 | Orientation | the device spectrum, JIT and AOT, generational collection |
| 2 | Concurrency | the event loop, isolates against async, goroutines, channels |
| 3 | Embedded | MCU anatomy, bare metal or an RTOS, TinyGo targets, I2C |
| 4 | Networking | MQTT topics, the three QoS levels, retained, the last will |
| 5 | BLE | GAP roles, GATT, notify or indicate, the permission flows |
| 6 | Sensors | what each sensor measures, rates, location accuracy modes |
| 7 | Background | Doze, standby buckets, WorkManager, iOS background modes |

**Revise rules and numbers.**

# Revision topics, Lectures 8 to 14

| L | Topic | Revise |
|---|---|---|
| 8 | Offline, part 1 | the local database, the outbox, delta sync with a cursor |
| 9 | Offline, part 2 | logical clocks, the four conflict strategies, CRDTs |
| 10 | Backends | serverless or a VPS, API design, deploying a Go binary |
| 11 | gRPC | the proto contract, stubs, the four call shapes, deadlines |
| 12 | Boundaries | channels, dart:ffi, what a call costs, Multiplatform |
| 13 | Rendering | the frame budget, the three trees, keys, const, lazy lists |
| 14 | Today | the spectrum, build modes, race to idle, the duty cycle |

**Every lecture can appear.** The paper is spread across the whole semester.

# Key numbers for the exam

**1/rate**

the frame budget: 8.3 ms at 120 Hz, 16.7 ms at 60 Hz

**8–16 GB**

RAM on a flagship phone, against 520 KB on an ESP32

**2 bytes**

the fixed MQTT header, against hundreds for HTTP

**0, 1, 2**

the MQTT QoS levels: at most, at least, exactly once

**4 orders**

of magnitude from deep sleep to radio transmit

**3 stages**

in a toolchain: front end, IR, back end

# Common mistakes

### Safe is not idempotent

GET is both. PUT and DELETE are idempotent but not safe, because they change state.

### Isolates

No shared mutable memory, so no data races. Ordering races are still possible.

### Rebuilds

Only dirty subtrees rebuild. Element reuse is decided by runtimeType, then key.

### StatelessWidget

It does not build once. It rebuilds whenever its parent rebuilds.

### QoS 1

At least once means duplicates arrive. Every handler must tolerate them.

### Device clocks

They cannot order events across two phones. Use a logical clock or a server version.

# Outdated statements

### ”Skia draws Flutter”

Impeller draws on iOS, Android and desktop. Skia is left for the web and old Android.

### ”The budget is 16 ms”

Only at 60 Hz. Most current phones run at 90 or 120 Hz, so state it as one over the refresh rate.

### ”The JSON bridge”

The bridgeless architecture replaced it. JavaScript calls C++ directly in React Native.

### ”CodePush for updates”

The hosted service closed in March 2025. The mainstream successor is Expo EAS Update.

### ”Enable bitcode”

Deprecated in Xcode 14, and the App Store no longer accepts it.

### ”Isolates cannot race”

They cannot race on memory. Messages can still interleave in an order you did not expect.

# Example question 1: build modes

A Flutter app is built for the store. Select every statement that is true.

- **A.** A debug build runs the Dart VM with a JIT, which is what makes hot reload possible.
- **B.** A release build ships an interpreter next to the compiled machine code.
- **C.** An iOS release build is compiled ahead of time because a third-party app may not make a writable page executable.
- **D.** Profile mode is the right build in which to measure frame times.

---

**Answer: A, C and D.** Only B is false: a release build contains neither an interpreter nor a JIT, which is why hot reload is unavailable in it.

# Example question 2: the frame budget

A phone's display refreshes at 120 Hz. Select every statement that is true.

- **A.** The budget for one frame is about 8.3 ms.
- **B.** A task that occupies the UI isolate for 25 ms overruns the budget of about three frames.
- **C.** The frame budget is 16 ms on every phone.
- **D.** A single missed frame is visible to the user as jank.

---

**Answer: A, B and D.** C is false because the budget is one divided by the refresh rate, and 16 ms is only the value at 60 Hz.

# Example question 3: MQTT

A sensor publishes to a broker over MQTT. Select every statement that is true.

- **A.** QoS 1 can deliver the same message more than once, so a handler must be idempotent.
- **B.** A retained message is delivered to a client that subscribes after it was published.
- **C.** The last will is published by the broker when a client disconnects unexpectedly.
- **D.** A shorter keep-alive always saves energy on a battery-powered device.

---

**Answer: A, B and C.** D is false: a shorter keep-alive means more traffic and a radio that is held awake more often, not less.

# Example question 4: offline-first

Two phones edit the same row while offline. Select every statement that is true.

- **A.** Writing the row and its outbox entry in one transaction keeps the local state and the queue consistent.
- **B.** The device wall clock is a safe way to order edits made on two different phones.
- **C.** A grow-only counter converges without coordination, because merging takes the maximum per replica.
- **D.** Last writer wins can silently discard a concurrent edit.

---

**Answer: A, C and D.** B is false: unsynchronized clocks can disagree by minutes, so ordering needs a logical clock or a server version.

# Example question 5: energy

A phone app syncs, and a sensor node reports over Wi-Fi. Select every true statement.

- **A.** Ten small transfers spread over a minute usually cost more energy than one batched transfer of the same bytes.
- **B.** On an OLED panel dark mode is a saving, while on an LCD the backlight is on regardless.
- **C.** Keeping a core busy at a low clock is more efficient than finishing quickly and returning to idle.
- **D.** On an ESP32-class device the radio, not the processor core, dominates the power budget.

**Answer: A, B and D.** C inverts race to idle: the deep low-power states are only reachable once nothing is keeping a core awake.

4

# Defense and presentation

Project milestones, the defense rubric, and the concept presentation

Contents

| | | Page |
|---|---|---|
| 1 | Compilation strategies | 3 |
| 2 | Energy at the platform level | 13 |
| 3 | Exam preparation | 19 |
| **4** | **Defense and presentation** | 31 |

# Project milestones

| Session | Milestone |
|---|---|
| 1 | team formed and the idea approved against the five requirements |
| 3 | the device streams a reading to a serial console |
| 5 | the app receives that reading over MQTT or BLE |
| 7 | offline mode with sync |
| 9 | gRPC, FFI or a platform channel integrated |
| 11 | the measurement done, and the polish pass |
| 13 | demo and defense |

**The defense is in week 13.** The last session shows what already works.

# Defense rubric

| Requirement | What proves it in the room |
|---|---|
| A device component | a TinyGo program on a real board or in Wokwi, reading at least one sensor |
| Device to phone | the app shows the live reading arriving over MQTT or BLE |
| Offline-first | airplane mode on, an edit, airplane mode off, the edit reaches the server |
| One boundary | gRPC with a generated Dart client, or FFI, or a platform channel |
| A measured claim | one number you measured, with the method and the conditions |

**All five are needed for approval.** The 15 points reflect how well each is shown and explained.

# Demo guidelines

### Start cold

Launch the app in front of the room, not a screen that has been open for an hour.

### Break it on purpose

Pull the cable, turn on airplane mode, kill the broker.

### Show the number

State the measured claim, then say how it was measured before anyone has to ask.

### Everyone speaks

Each member presents the part they wrote and answers questions about it.

### Bring a fallback

A short recording of the demo, in case the room's network or the hardware fails you.

### Say what is next

One sentence about a known limitation.

# Concept presentations

- Teams of at most three. One topic, claimed from the shared sheet, and no two teams take the same one.
- About 20 minutes of slides, plus an optional demo. Every member presents a part of it.
- Materials go to Moodle.

**What earns the point**

A topic you can explain from something you built or ran, sources named on the slides, and a clear statement of what the technology is not good for.

**Choose a topic you can show, not only describe.**

# Summary

1. Every runtime decides when to translate. Every platform kept moving that moment earlier.
2. Energy follows one rule at both scales. Do the work, then let the hardware sleep.
3. Learn the rules with their conditions and the numbers with their units.

**Further reading**

docs.flutter.dev/perf · Baseline profiles · dart.dev · tinygo.org

# Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel